

LocalBuka Build-from-Scratch Guide

This is a code-first guide. Follow the steps in order and type the shown code into the named files. The explanations state why each implementation exists and how the pieces work together.

Step 1 — Create the package and install dependencies

Create the folders shown below. The two `__init__.py` files allow Python to run the project as modules from the repository root. Install the dependencies, then create a root `.env` file with your two real API keys. Do not place the key values in source code or commit `.env`.

Code to create: Project structure and setup commands

```
localbuka_case_study/  
  __init__.py  
  localbuka/  
    __init__.py  
    models.py  
    data.py  
    embeddings.py  
    pine_cone.py  
    recommender.py  
    assistant.py  
    cli.py  
  tests/  
    test_localbuka.py  
  ingest.py  
  run_examples.py  
  requirements.txt  
  
python3 -m pip install google-genai==2.20.0 pinecone==9.1.0 python-dotenv==1.2.2 reportlab==5.0.1  
  
# .env at the AI_Engineering root – never commit this file  
GEMINI_API_KEY=your_gemini_key  
PINECONE_API_KEY=your_pinecone_key
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 2 — Define the data models

Write this file first because the rest of the application shares these objects. Dish represents an item on a menu. UserPreferences represents what the user asked for. Recommendation represents an item after retrieval. PRICE_ORDER makes price comparisons explicit; the two validation lists prevent unsupported filters from being sent to the services.

Code to create: localbuka/models.py

```
"""Simple data containers used by the LocalBuka prototype."""

from dataclasses import dataclass, field

# Numbers make it easy to compare the three price bands.
PRICE_ORDER = {"budget": 1, "mid": 2, "premium": 3}
VALID_DIETARY_TAGS = ["vegetarian", "vegan", "halal", "gluten-free"]
VALID_SPICE_LEVELS = ["mild", "medium", "hot"]

@dataclass
class Dish:
    """Stores the details of one dish in the sample catalogue."""

    id: str
    restaurant: str
    city: str
    cuisine: str
    name: str
    price_band: str
    dietary_tags: list[str]
    spice_level: str
    popularity: int
    description: str

@dataclass
class UserPreferences:
    """Stores the optional preferences collected from a user."""

    cuisines: list[str] = field(default_factory=list)
    max_price_band: str = None
    dietary_restrictions: list[str] = field(default_factory=list)
    spice_preference: str = None
    cities: list[str] = field(default_factory=list)
    past_order_dish_ids: list[str] = field(default_factory=list)

@dataclass
class Recommendation:
    """Stores one ranked dish and the plain-English reasons for its score."""

    dish: Dish
    score: float
    reasons: list[str]
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 3 — Add the Nigeria-first sample catalogue

This file gives the prototype its data. The dish helper makes the 24 records easier to read. Store city, cuisine, price, dietary tags, and spice as structured data because they are safety/product constraints. Keep the description because it provides semantic context for Gemini embeddings.

Code to create: localbuka/data.py

```
"""A deliberately small, hand-curated Nigeria-first food catalogue."""

from .models import Dish

def dish(
    id: str, restaurant: str, city: str, cuisine: str, name: str, price_band: str,
    dietary_tags: list[str], spice_level: str, popularity: int, description: str,
) -> Dish:
    return Dish(id, restaurant, city, cuisine, name, price_band, dietary_tags, spice_level, popularity, description)

CATALOGUE = [
    dish("lag-01", "Buka on Adeola", "Lagos", "nigerian", "Smoky Jollof Rice & Grilled Chicken", "mid", ["halal"], "medium", 95, "Party-
    dish("lag-02", "Eko Plant Kitchen", "Lagos", "nigerian", "Asaro with Fried Plantain", "budget", ["vegan", "vegetarian", "halal", "g
    dish("lag-03", "Lagos Lagoon Seafood", "Lagos", "nigerian", "Peppered Catfish & Yam", "premium", ["halal", "gluten-free"], "hot", 88
    dish("lag-04", "Suya Junction", "Lagos", "nigerian", "Beef Suya Wrap", "budget", ["halal"], "hot", 91, "Spiced skewered beef, onions
    dish("lag-05", "The Continental Plate", "Lagos", "continental", "Creamy Mushroom Pasta", "premium", ["vegetarian"], "mild", 75, "Tag
    dish("lag-06", "Iya Tinu's Kitchen", "Lagos", "nigerian", "Efo Riro & Amala", "mid", ["halal", "gluten-free"], "hot", 90, "Rich spin
    dish("abu-01", "Wuse Buka House", "Abuja", "nigerian", "Tuwo Shinkafa & Miyan Kuka", "budget", ["halal", "gluten-free"], "medium", 8
    dish("abu-02", "Capital Vegan Table", "Abuja", "nigerian", "Ofada Rice & Ayamase Mushrooms", "mid", ["vegan", "vegetarian", "halal",
    dish("abu-03", "Jabi Grill", "Abuja", "continental", "Peri-Peri Chicken & Rice", "mid", ["halal", "gluten-free"], "hot", 87, "Flame-
    dish("abu-04", "Northern Spice Hub", "Abuja", "nigerian", "Kilishi Platter", "premium", ["halal", "gluten-free"], "hot", 72, "Air-dr
    dish("ph-01", "Creekside Pot", "Port Harcourt", "nigerian", "Fisherman Soup & Pounded Yam", "premium", ["halal", "gluten-free"], "h
    dish("ph-02", "Garden City Buka", "Port Harcourt", "nigerian", "Bole & Fish", "budget", ["halal", "gluten-free"], "medium", 93, "Ro
    dish("ph-03", "Rivers Veggie Bowl", "Port Harcourt", "nigerian", "Beans & Dodo Bowl", "budget", ["vegan", "vegetarian", "halal", "g
    dish("iba-01", "Oyo Heritage Kitchen", "Ibadan", "nigerian", "Abula Combo", "budget", ["halal", "gluten-free"], "medium", 84, "Amala
    dish("iba-02", "Bodija Bowl", "Ibadan", "nigerian", "Afang Soup & Fufu", "mid", ["halal", "gluten-free"], "medium", 77, "Leafy afang
    dish("enu-01", "Coal City Kitchen", "Enugu", "nigerian", "Oha Soup & Pounded Yam", "mid", ["halal", "gluten-free"], "medium", 85, "A
    dish("enu-02", "Nsukka Rice Bar", "Enugu", "nigerian", "Coconut Jollof & Plantain", "budget", ["vegan", "vegetarian", "halal", "glut
    dish("kan-01", "Kano Kilishi Court", "Kano", "nigerian", "Masa & Miyan Taushe", "budget", ["vegetarian", "halal"], "mild", 76, "Rice
    dish("kan-02", "Arewa Flame Grill", "Kano", "nigerian", "Chicken Suya & Kuli-Kuli", "mid", ["halal", "gluten-free"], "hot", 89, "Spi
    dish("ben-01", "Benin Pepper Pot", "Benin City", "nigerian", "Banga Soup & Starch", "mid", ["halal", "gluten-free"], "hot", 83, "Pa
    dish("lag-07", "Marina Mezze", "Lagos", "continental", "Falafel Bowl", "mid", ["vegan", "vegetarian", "halal"], "mild", 74, "Falafel
    dish("lag-08", "Yaba Morning Buka", "Lagos", "nigerian", "Akara & Pap Breakfast Bowl", "budget", ["vegan", "vegetarian", "halal", "g
    dish("lag-09", "Lekki Green Pot", "Lagos", "nigerian", "Spicy Beans & Plantain", "budget", ["vegan", "vegetarian", "halal", "gluten-
    dish("lag-10", "Sabo Bistro", "Lagos", "continental", "Roasted Vegetable Pizza", "premium", ["vegetarian"], "mild", 71, "Stone-baked
]
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 4 — Create the Gemini embedding client

GeminiEmbedder is the only class that calls the Gemini embedding API. The 768-dimensional output is chosen deliberately and must match the Pinecone index dimension in the next step. `embed_texts` embeds each record in a loop to guarantee one vector per dish. `embed_one_text` returns one vector for a search query, preventing an accidental list-of-vectors error.

Code to create: `localbuka/embeddings.py`

```
"""Gemini embedding helper for LocalBuka's restaurant and dish text."""

import os
from pathlib import Path

from dotenv import load_dotenv
from google import genai
from google.genai import types

class GeminiEmbedder:
    """This Embedder creates 768-number embeddings using the Gemini API."""

    MODEL_NAME = "gemini-embedding-2"
    DIMENSION = 768

    def __init__(self):
        root_folder = Path(__file__).resolve().parents[2]
        load_dotenv(root_folder / ".env")
        api_key = os.getenv("GEMINI_API_KEY")
        if not api_key:
            raise ValueError("GEMINI_API_KEY is missing. Add it to the root .env file.")
        self.client = genai.Client(api_key=api_key)

    def embed_texts(self, texts):
        """This method sends one or more text strings to Gemini and returns their vectors."""
        vectors = []
        for text in texts:
            response = self.client.models.embed_content(
                model=self.MODEL_NAME,
                contents=text,
                config=types.EmbedContentConfig(output_dimensionality=self.DIMENSION),
            )
            vectors.append(response.embeddings[0].values)
        return vectors

    def embed_one_text(self, text):
        return self.embed_texts([text])[0]
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 5 — Create the Pinecone vector-store wrapper

This class owns index setup, upserts, and searches. `_create_index_if_needed` makes the operation safe to run more than once. The `upsert` method writes the Gemini vector plus structured metadata. The `search` method uses city, cuisine, and maximum price as deterministic Pinecone filters; semantic similarity then ranks the remaining candidates. Dietary tags are intentionally checked again later in Python.

Code to create: `localbuka/pine_cone.py`

```
"""Pinecone index setup, dish ingestion, and semantic-search helpers."""

import os
import time
from pathlib import Path

from dotenv import load_dotenv
from pinecone import Pinecone, ServerlessSpec

class PineconeRestaurantIndex:
    """Stores LocalBuka dish embeddings and retrieves semantically similar dishes"""

    INDEX_NAME = "localbuka-gemini-768"
    NAMESPACE = "localbuka-dishes"
    DIMENSION = 768

    def __init__(self):
        root_folder = Path(__file__).resolve().parents[2]
        load_dotenv(root_folder / ".env")
        api_key = os.getenv("PINECONE_API_KEY")
        if not api_key:
            raise ValueError("PINECONE_API_KEY is missing. Add it to the root .env file.")
        self.pinecone = Pinecone(api_key=api_key)
        self._create_index_if_needed()
        self.index = self.pinecone.Index(self.INDEX_NAME)

    def _create_index_if_needed(self):
        if self.pinecone.has_index(self.INDEX_NAME):
            return
        self.pinecone.create_index(
            name=self.INDEX_NAME,
            dimension=self.DIMENSION,
            metric="cosine",
            spec=ServerlessSpec(cloud="aws", region="us-east-1"),
            deletion_protection="disabled",
        )
        while not self.pinecone.describe_index(self.INDEX_NAME).status["ready"]:
            time.sleep(1)

    def upsert_dishes(self, dishes, vectors):
        """Store one Gemini vector and safe filter metadata for every dish."""
        records = []
        for dish, vector in zip(dishes, vectors):
            metadata = {
                "restaurant": dish.restaurant,
                "city": dish.city,
                "cuisine": dish.cuisine,
                "name": dish.name,
                "price_band": dish.price_band,
                "price_rank": self._price_rank(dish.price_band),
                "dietary_tags": dish.dietary_tags,
                "spice_level": dish.spice_level,
                "popularity": dish.popularity,
                "description": dish.description,
            }
            records.append({"id": dish.id, "values": vector, "metadata": metadata})
        self.index.upsert(vectors=records, namespace=self.NAMESPACE)

    def search(self, query_vector, preferences, top_k):
        """Query Pinecone and apply city/price metadata filters before ranking."""
        metadata_filter = {}
        if preferences.cities:
            metadata_filter["city"] = {"$in": preferences.cities}
        if preferences.cuisines:
            metadata_filter["cuisine"] = {"$in": preferences.cuisines}
        if preferences.max_price_band:
            metadata_filter["price_rank"] = {"$lte": self._price_rank(preferences.max_price_band)}
        response = self.index.query(
            vector=query_vector,
            top_k=top_k,
            namespace=self.NAMESPACE,
            include_metadata=True,
            filter=metadata_filter or None,
        )
        results = []
        for match in response.matches:
            results.append({"id": match.id, "score": match.score})
```

```
        return results

    def _price_rank(self, price_band):
        prices = {"budget": 1, "mid": 2, "premium": 3}
        return prices[price_band]
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 6 — Build ingestion

dish_to_text turns each structured dish into one rich text description for Gemini. main creates the two service clients, builds a text for every catalogue record, embeds those texts, and upserts the results. Run this one-time setup after creating or changing data.py. In production, trigger this only for changed dishes rather than re-embedding the entire catalogue.

Code to create: ingest.py

```
"""Create Gemini embeddings for the sample dishes and store them in Pinecone."""

from .localbuka.data import CATALOGUE
from .localbuka.embeddings import GeminiEmbedder
from .localbuka.pine_cone import PineconeRestaurantIndex

def dish_to_text(dish):
    """Build the searchable sentence Gemini will convert into an embedding."""
    dietary_tags = ", ".join(dish.dietary_tags)
    return (
        f"{dish.name}. Restaurant: {dish.restaurant}. City: {dish.city}. "
        f"Cuisine: {dish.cuisine}. Price: {dish.price_band}. "
        f"Dietary tags: {dietary_tags}. Spice: {dish.spice_level}. "
        f"Description: {dish.description}"
    )

def main():
    embedder = GeminiEmbedder()
    restaurant_index = PineconeRestaurantIndex()
    dish_texts = []
    for dish in CATALOGUE:
        dish_texts.append(dish_to_text(dish))
    vectors = embedder.embed_texts(dish_texts)
    restaurant_index.upsert_dishes(CATALOGUE, vectors)
    print(f"Stored {len(CATALOGUE)} Gemini embeddings in the Pinecone index.")

if __name__ == "__main__":
    main()
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 7 — Build the recommendation workflow

Recommender coordinates the search. Constructor injection of catalogue, embedder, and restaurant_index is deliberate: it allows tests to use fake services. recommend validates filters, enriches the query with prior-order names, embeds the query, asks Pinecone for extra candidates, then performs a final dietary check. The display score is primarily semantic similarity, with a very small popularity tiebreaker.

Code to create: localbuka/recommender.py

```
"""Gemini embedding and Pinecone retrieval logic for LocalBuka."""

from .models import PRICE_ORDER, Recommendation, VALID_DIETARY_TAGS, VALID_SPICE_LEVELS

class Recommender:
    """Find semantically relevant dishes while enforcing user safety constraints."""

    def __init__(self, catalogue, embedder, restaurant_index):
        self.catalogue = catalogue
        self.embedder = embedder
        self.restaurant_index = restaurant_index
        self.dishes_by_id = {}
        for dish in catalogue:
            self.dishes_by_id[dish.id] = dish

    def recommend(self, query_text, preferences, limit=5):
        """Embed the request, retrieve Pinecone matches, then double-check diet tags."""
        self._validate(preferences, limit)
        query_with_history = self._add_order_history(query_text, preferences.past_order_dish_ids)
        query_vector = self.embedder.embed_one_text(query_with_history)
        matches = self.restaurant_index.search(query_vector, preferences, top_k=limit * 5)

        recommendations = []
        for match in matches:
            dish = self.dishes_by_id.get(match["id"])
            if not dish:
                continue
            if not self._meets_dietary_requirements(dish, preferences.dietary_restrictions):
                continue
            score = round((match["score"] * 100) + (dish.popularity * 0.05), 1)
            reasons = self._build_reasons(dish, preferences, match["score"])
            recommendations.append(Recommendation(dish, score, reasons))
            if len(recommendations) == limit:
                break
        return recommendations

    def _validate(self, preferences, limit):
        if limit < 1:
            raise ValueError("limit must be at least 1")
        if preferences.max_price_band and preferences.max_price_band not in PRICE_ORDER:
            raise ValueError("max_price_band must be budget, mid, or premium")
        for restriction in preferences.dietary_restrictions:
            if restriction not in VALID_DIETARY_TAGS:
                raise ValueError(f"Unsupported dietary restriction: {restriction}")
        if preferences.spice_preference and preferences.spice_preference not in VALID_SPICE_LEVELS:
            raise ValueError("spice_preference must be mild, medium, or hot")

    def _add_order_history(self, query_text, dish_ids):
        history_names = []
        for dish_id in dish_ids:
            dish = self.dishes_by_id.get(dish_id)
            if dish:
                history_names.append(dish.name)
        if not history_names:
            return query_text
        return query_text + ". The user previously enjoyed: " + ", ".join(history_names)

    def _meets_dietary_requirements(self, dish, restrictions):
        for restriction in restrictions:
            if restriction not in dish.dietary_tags:
                return False
        return True

    def _build_reasons(self, dish, preferences, similarity_score):
        reasons = [f"semantic match score: {similarity_score:.3f}"]
        if preferences.cities:
            reasons.append(f"available in {dish.city}")
        if preferences.max_price_band:
            reasons.append(f"within your {preferences.max_price_band} budget")
        if preferences.dietary_restrictions:
            reasons.append("meets your dietary requirement")
        if preferences.spice_preference and dish.spice_level == preferences.spice_preference:
            reasons.append(f"matches your {dish.spice_level} spice preference")
        return reasons
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 8 — Build the conversational parser

FoodAssistant is bounded rather than generative. The term dictionaries translate a small known vocabulary such as cheap to budget and spicy to hot. `reply` guards against unavailable restaurant facts, `parse_preferences` creates a `UserPreferences` object, and the helpers keep the parsing readable. This design is safer than allowing an LLM to invent hours, delivery coverage, or allergen claims.

Code to create: `localbuka/assistant.py`

```
"""A bounded, data-grounded conversational layer for food discovery."""

from .models import PRICE_ORDER, UserPreferences
from .recommender import Recommender

class FoodAssistant:
    """Convert limited food-discovery language into catalogue-backed suggestions."""

    CUISINE_TERMS = {"nigerian": "nigerian", "local": "nigerian", "continental": "continental", "pasta": "continental", "falafel": "continental"}
    DIET_TERMS = {"vegan": "vegan", "vegetarian": "vegetarian", "halal": "halal", "gluten-free": "gluten-free", "gluten free": "gluten-free"}
    PRICE_TERMS = {"cheap": "budget", "affordable": "budget", "budget": "budget", "mid-range": "mid", "mid range": "mid", "premium": "premium"}
    SPICE_TERMS = {"spicy": "hot", "hot": "hot", "medium spice": "medium", "mild": "mild", "not spicy": "mild"}

    def __init__(self, recommender: Recommender):
        self.recommender = recommender
        self._cities = {}
        for dish in recommender.catalogue:
            self._cities[dish.city.lower()] = dish.city

    def reply(self, message, location=None):
        """Produce a short answer containing only catalogue facts.

        The prototype intentionally does not use an LLM, so it cannot invent opening
        times, delivery coverage, prices, nutrition values, or restaurant claims.
        """
        normalized = message.strip().lower()
        if not normalized:
            return "Tell me what you feel like eating, for example: 'cheap spicy Nigerian food in Lagos'."
        operational_terms = ["opening", "hours", "address", "phone", "reservation", "delivery"]
        for term in operational_terms:
            if term in normalized:
                return "I only have the sample menu catalogue, so I can't verify operational details. I can help you find a dish by city."
        preferences = self.parse_preferences(normalized, location)
        if "near me" in normalized and not preferences.cities:
            return "Which Nigerian city are you in? I currently support Lagos, Abuja, Port Harcourt, Ibadan, Enugu, Kano, and Benin City."
        results = self.recommender.recommend(normalized, preferences, limit=3)
        if not results:
            return "I couldn't find a compatible dish in this sample catalogue. Try relaxing one preference or choose another city."
        lines = ["Here are the best matches from the sample catalogue:"]
        for index, result in enumerate(results, start=1):
            dish = result.dish
            lines.append(f"{index}. {dish.name} – {dish.restaurant}, {dish.city} ({dish.price_band}, {dish.spice_level}). {result.reason}")
        return "\n".join(lines)

    def parse_preferences(self, message, location=None):
        """Extract only terms supported by the catalogue; unknown text is ignored."""
        cities = []
        for city_name, city_value in self._cities.items():
            if self._contains(message, city_name):
                cities.append(city_value)
        if not cities and location and "near me" in message:
            canonical = self._cities.get(location.strip().lower())
            if canonical:
                cities.append(canonical)
        cuisines = self._matched_values(message, self.CUISINE_TERMS)
        dietary = self._matched_values(message, self.DIET_TERMS)
        price = self._best_price(message)
        spice = self._best_spice(message)
        return UserPreferences(self._remove_duplicates(cuisines), price, self._remove_duplicates(dietary), spice, self._remove_duplicates(locations))

    @staticmethod
    def _contains(message, term):
        """Use basic substring matching because the supported vocabulary is small."""
        return term in message

    def _matched_values(self, message, terms):
        matches = []
        for term, value in terms.items():
            if self._contains(message, term):
                matches.append(value)
        return matches

    def _best_price(self, message):
        matches = self._matched_values(message, self.PRICE_TERMS)
        if not matches:
            return None
        cheapest_match = matches[0]
```

```

    for price_band in matches:
        if PRICE_ORDER[price_band] < PRICE_ORDER[cheapest_match]:
            cheapest_match = price_band
    return cheapest_match

def _best_spice(self, message):
    if self._contains(message, "not spicy"):
        return "mild"
    matches = self._matched_values(message, self.SPICE_TERMS)
    return matches[0] if matches else None

def _remove_duplicates(self, items):
    unique_items = []
    for item in items:
        if item not in unique_items:
            unique_items.append(item)
    return unique_items

```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 9 — Add the command-line entry point

main creates objects in dependency order: GeminiEmbedder, PineconeRestaurantIndex, Recommender, then FoodAssistant. It asks for a city once so near me can become an explicit filter. The loop reads user requests until quit or exit is entered.

Code to create: localbuka/cli.py

```
"""Command-line entry point for the LocalBuka assistant."""

from .assistant import FoodAssistant
from .data import CATALOGUE
from .embeddings import GeminiEmbedder
from .pine_cone import PineconeRestaurantIndex
from .recommender import Recommender

def main() -> None:
    embedder = GeminiEmbedder()
    restaurant_index = PineconeRestaurantIndex()
    assistant = FoodAssistant(Recommender(CATALOGUE, embedder, restaurant_index))
    print("Welcome to LocalBuka . 'quit' to leave.")
    location = input("Type your city: ").strip() or None
    while True:
        try:
            message = input("Type what you want to eat: ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nGoodbye!")
            return
        if message.lower() == "quit" or message.lower() == "exit":
            print("Thanks for using our platform, bye!")
            return
        print(f"LocalBuka: {assistant.reply(message, location)}")

if __name__ == "__main__":
    main()
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 10 — Add three live example scenarios

This script demonstrates that different preferences produce different recommendations. It calls the real embedding and vector services, so the index must already be ingested and keys must be configured. Use it as a live demo after the unit tests.

Code to create: run_examples.py

```
"""Render the required recommendation examples without API keys or network access."""

from .localbuka.data import CATALOGUE
from .localbuka.embeddings import GeminiEmbedder
from .localbuka.models import UserPreferences
from .localbuka.pine_cone import PineconeRestaurantIndex
from .localbuka.recommender import Recommender

EXAMPLES = {
    "Ada – budget-conscious vegan in Lagos": UserPreferences(
        cuisines=["nigerian"], max_price_band="budget", dietary_restrictions=["vegan"],
        spice_preference="medium", cities=["Lagos"],
    ),
    "Tunde – hot, halal food in Abuja": UserPreferences(
        cuisines=["nigerian"], max_price_band="mid", dietary_restrictions=["halal"],
        spice_preference="hot", cities=["Abuja"], past_order_dish_ids=["abu-01", "kan-01"],
    ),
    "Chioma – mild continental choice in Lagos": UserPreferences(
        cuisines=["continental"], max_price_band="premium", dietary_restrictions=["vegetarian"],
        spice_preference="mild", cities=["Lagos"],
    ),
}

def main() -> None:
    embedder = GeminiEmbedder()
    restaurant_index = PineconeRestaurantIndex()
    recommender = Recommender(CATALOGUE, embedder, restaurant_index)
    for label, preferences in EXAMPLES.items():
        print(f"## {label}")
        query_parts = preferences.cuisines + preferences.dietary_restrictions + preferences.cities
        if preferences.spice_preference:
            query_parts.append(preferences.spice_preference)
        query_parts.append("food")
        query_text = " ".join(query_parts)
        for position, result in enumerate(recommender.recommend(query_text, preferences, limit=3), 1):
            print(f"{position}. {result.dish.name} | {result.dish.restaurant}, {result.dish.city} | score {result.score} | {'; '.join(re)}")

if __name__ == "__main__":
    main()
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 11 — Add offline tests

The FakeEmbedder and FakeRestaurantIndex have the same methods as the real services but make no network calls. This isolates application logic from cost and service availability. These tests protect parsing, query construction, dietary rechecking, and safe refusals.

Code to create: tests/test_localbuka.py

```
import unittest

from localbuka.assistant import FoodAssistant
from localbuka.data import CATALOGUE
from localbuka.models import UserPreferences
from localbuka.recommender import Recommender

class FakeEmbedder:
    """Avoids paid Gemini calls while testing the local application logic."""

    def __init__(self):
        self.last_text = ""

    def embed_one_text(self, text):
        self.last_text = text
        return [0.1, 0.2, 0.3]

class FakeRestaurantIndex:
    """Returns predictable Pinecone-like results without a network call."""

    def __init__(self):
        self.last_preferences = None

    def search(self, query_vector, preferences, top_k):
        self.last_preferences = preferences
        return [
            {"id": "lag-02", "score": 0.95},
            {"id": "lag-04", "score": 0.90},
            {"id": "lag-01", "score": 0.85},
        ]

class LocalBukaTests(unittest.TestCase):
    def setUp(self):
        self.embedder = FakeEmbedder()
        self.restaurant_index = FakeRestaurantIndex()
        self.recommender = Recommender(CATALOGUE, self.embedder, self.restaurant_index)
        self.assistant = FoodAssistant(self.recommender)

    def test_catalogue_exceeds_required_size(self):
        self.assertGreaterEqual(len(CATALOGUE), 20)

    def test_dietary_requirements_are_checked_after_vector_search(self):
        preferences = UserPreferences(dietary_restrictions=["vegan"])
        results = self.recommender.recommend("vegan food", preferences)
        self.assertTrue(results)
        self.assertTrue(all("vegan" in item.dish.dietary_tags for item in results))

    def test_recommender_sends_query_text_to_the_embedder(self):
        preferences = UserPreferences(cities=["Lagos"])
        self.recommender.recommend("cheap food near me", preferences)
        self.assertEqual(self.embedder.last_text, "cheap food near me")
        self.assertEqual(self.restaurant_index.last_preferences.cities, ["Lagos"])

    def test_past_order_names_are_added_to_the_embedding_query(self):
        preferences = UserPreferences(past_order_dish_ids=["lag-01"])
        self.recommender.recommend("something similar", preferences)
        self.assertIn("Smoky Jollof Rice", self.embedder.last_text)

    def test_assistant_parses_near_me_using_known_location(self):
        parsed = self.assistant.parse_preferences("I want something spicy and cheap near me", "Lagos")
        self.assertEqual(parsed.max_price_band, "budget")
        self.assertEqual(parsed.spice_preference, "hot")
        self.assertEqual(parsed.cities, ["Lagos"])

    def test_assistant_requests_location_when_near_me_is_ambiguous(self):
        reply = self.assistant.reply("I want something spicy and cheap near me")
        self.assertIn("Which Nigerian city", reply)

    def test_assistant_does_not_invent_operational_information(self):
        reply = self.assistant.reply("What are the opening hours of Buka on Adeola?")
        self.assertIn("can't verify operational details", reply)

if __name__ == "__main__":
    unittest.main()
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.

Step 12 — Run the complete application

Run the commands in this order from the AI_Engineering repository root. Ingestion creates or refreshes the 24 vectors. Tests prove the local logic without APIs. The examples and CLI use Gemini and Pinecone. If you see a dimension error, confirm that Gemini's output_dimensionality and the Pinecone index dimension are both 768, and that you passed one vector rather than a list of vectors.

Code to create: Verification commands

```
# One-time or after changing localbuka/data.py
python3 -m localbuka_case_study.ingest

# Offline tests; no Gemini or Pinecone API calls
cd localbuka_case_study
python3 -m unittest discover -s tests -v
cd ..

# Three live scenarios
python3 -m localbuka_case_study.run_examples

# Interactive chat
python3 -m localbuka_case_study.localbuka.cli

# Example interaction
Type your city: Lagos
Type what you want to eat: I want something spicy and cheap near me

# Safety check
Type what you want to eat: What are the opening hours?
```

Reasoning checkpoint: Do not move to the next step until this code is saved in the stated location. The next implementation depends on the names and interfaces defined here.